3D Perception Obstacle Avoidance

3D Perception Obstacle Avoidance

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis
5. Parameter Modification

1. Course Content

- This course explains how to use a depth camera to detect obstacles in the environment and let the robot arm automatically avoid obstacles during motion planning.

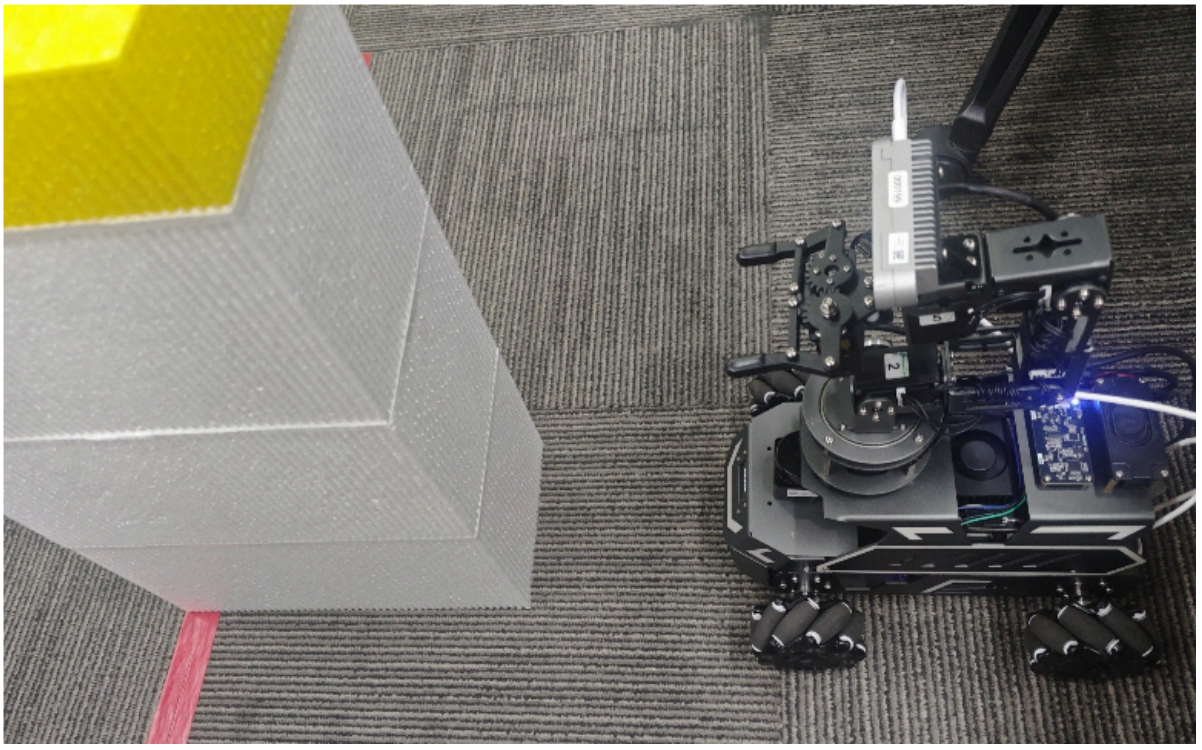
2. Preparation

```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
rviz_config:=demo 3d_perception:=true
```

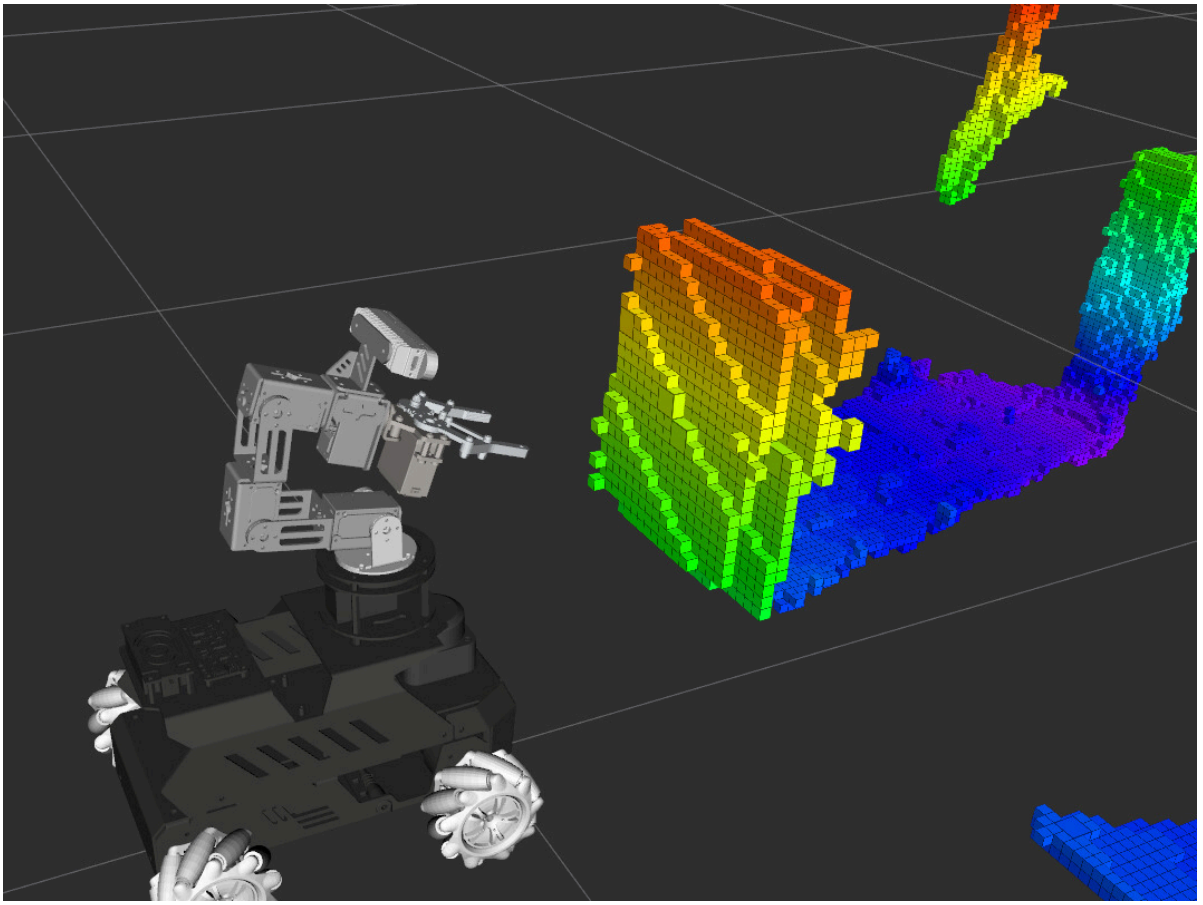
- `sim2real:=true`: Enables the simulation-reality linkage mode; the trajectory planned in RViz will synchronously drive the real robot arm.
- `rviz_config:=demo`: Uses the RViz configuration file of demo mode, displaying visualization elements such as the planned trajectory and collision objects.
- `3d_perception:=true`: Enables the 3D perception feature, loads the depth camera sensor configuration, and converts the camera point cloud data into an occupancy grid map (Octomap) for obstacle detection and avoidance during planning.

3. Case Demonstration

- Place an obstacle in front of the car, not too close, at least 20 cm away from the depth camera to avoid the camera's blind zone.



- In RViz, the depth camera's point cloud will show the position and approximate shape of the obstacle through the occupancy voxel grid.



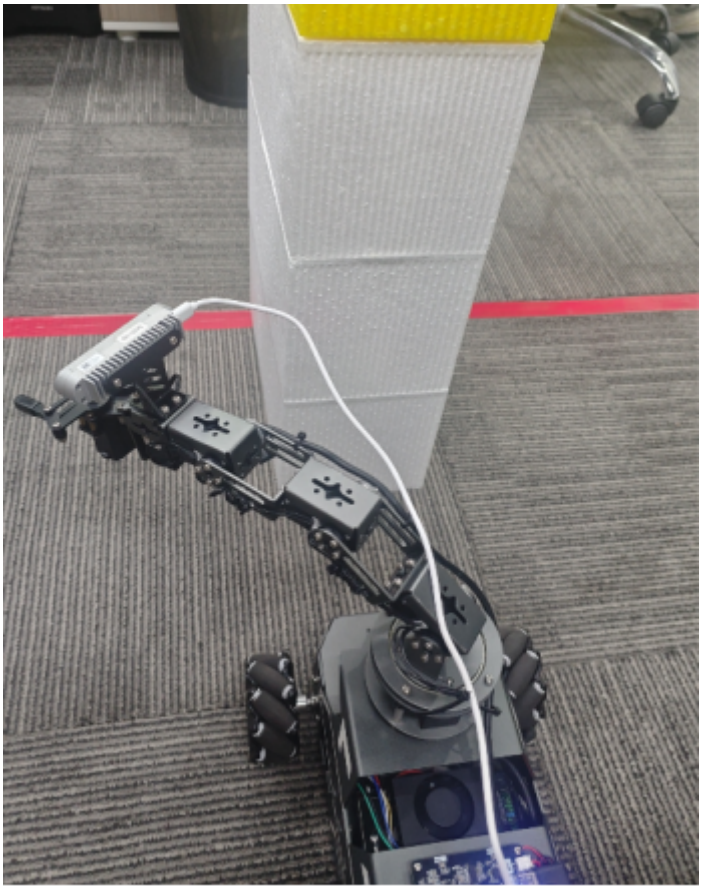
- At this point, when controlling the robot arm motion, the planned trajectory will preferentially avoid the obstacle position, for example, moving from one pose A to another pose B.

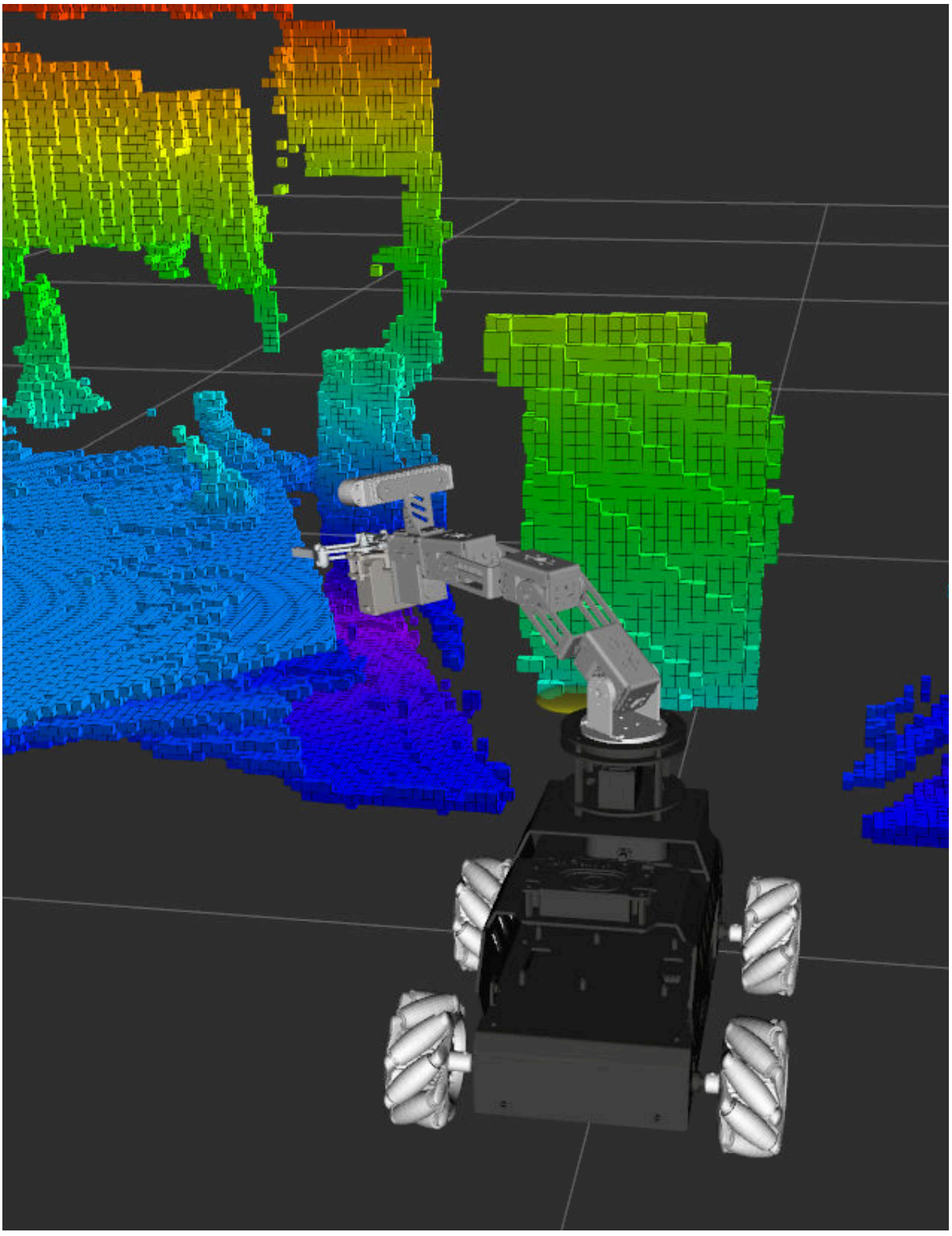
> **Important**

> The target_joints in the following motion commands can be specified and modified by yourself. Here, joint-space planning is used as an example for demonstration. If the pose in the example is too close to the obstacle and causes planning to fail, you can change the value of joint 1 to gradually move away from the obstacle.

- Move to pose A

```
ros2 run moveit_demo set_target_joints --ros-args -p target_joints:="
[40.0,34.0,56.0,81.0,90.0]"
```

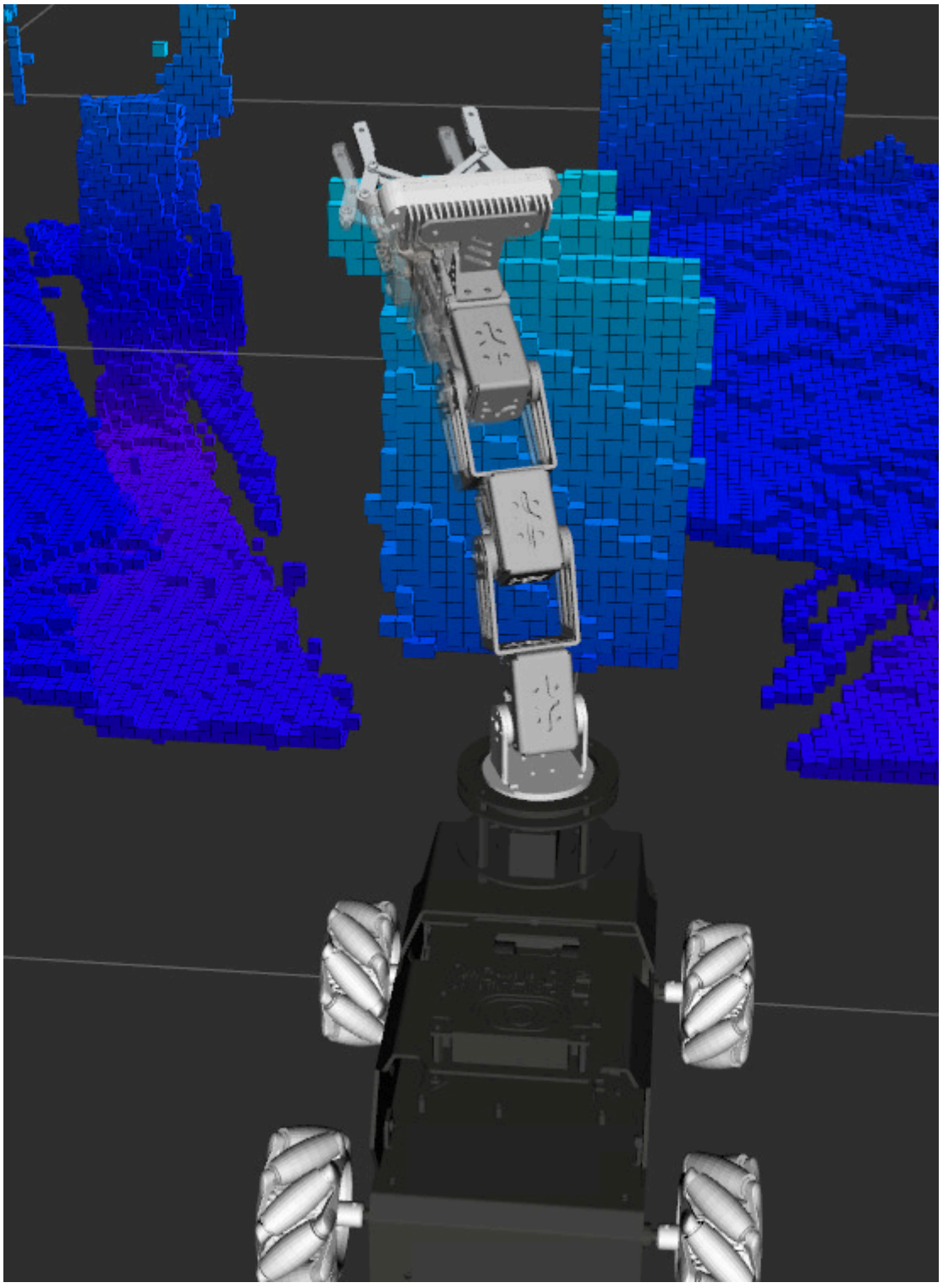


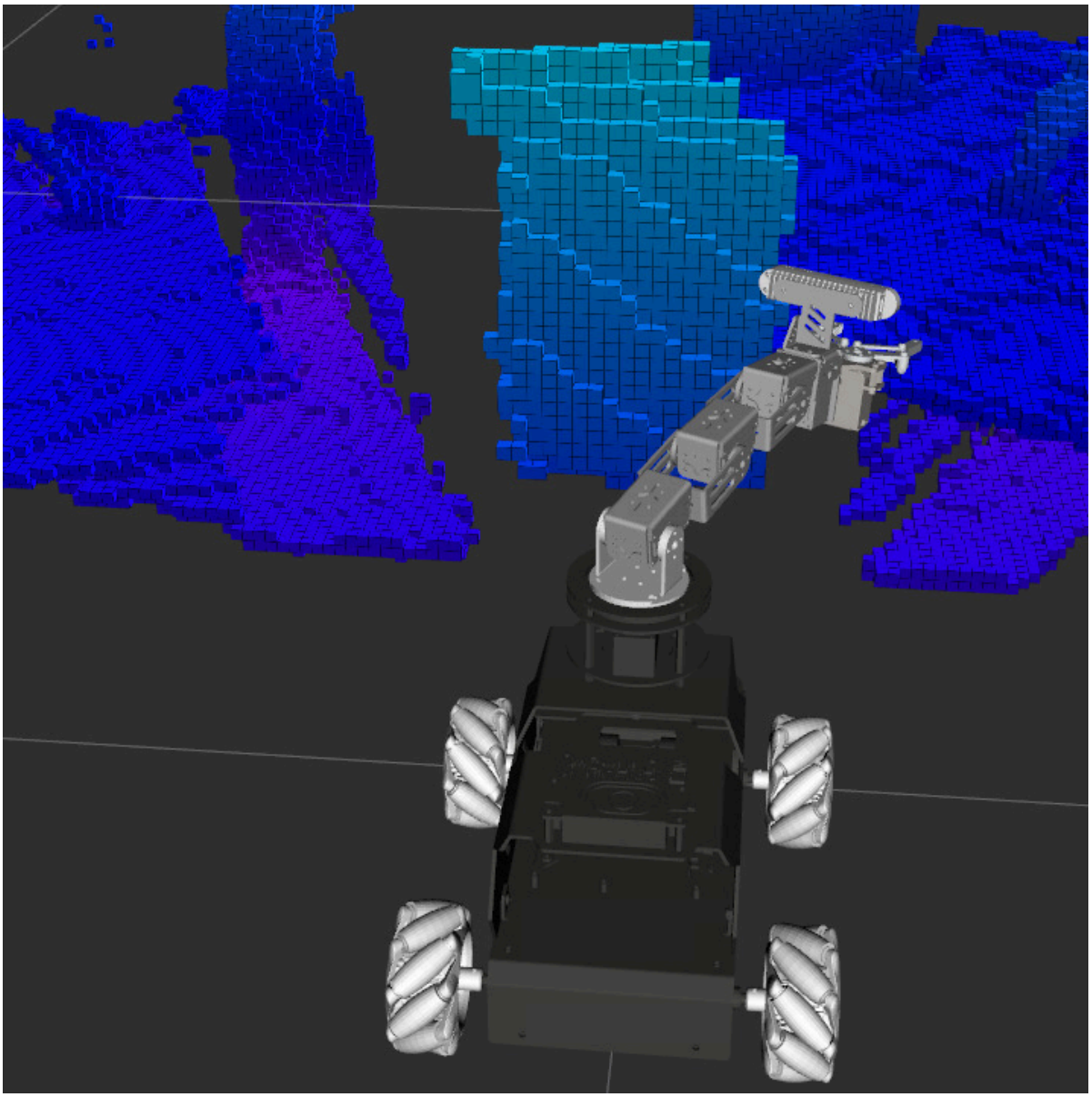


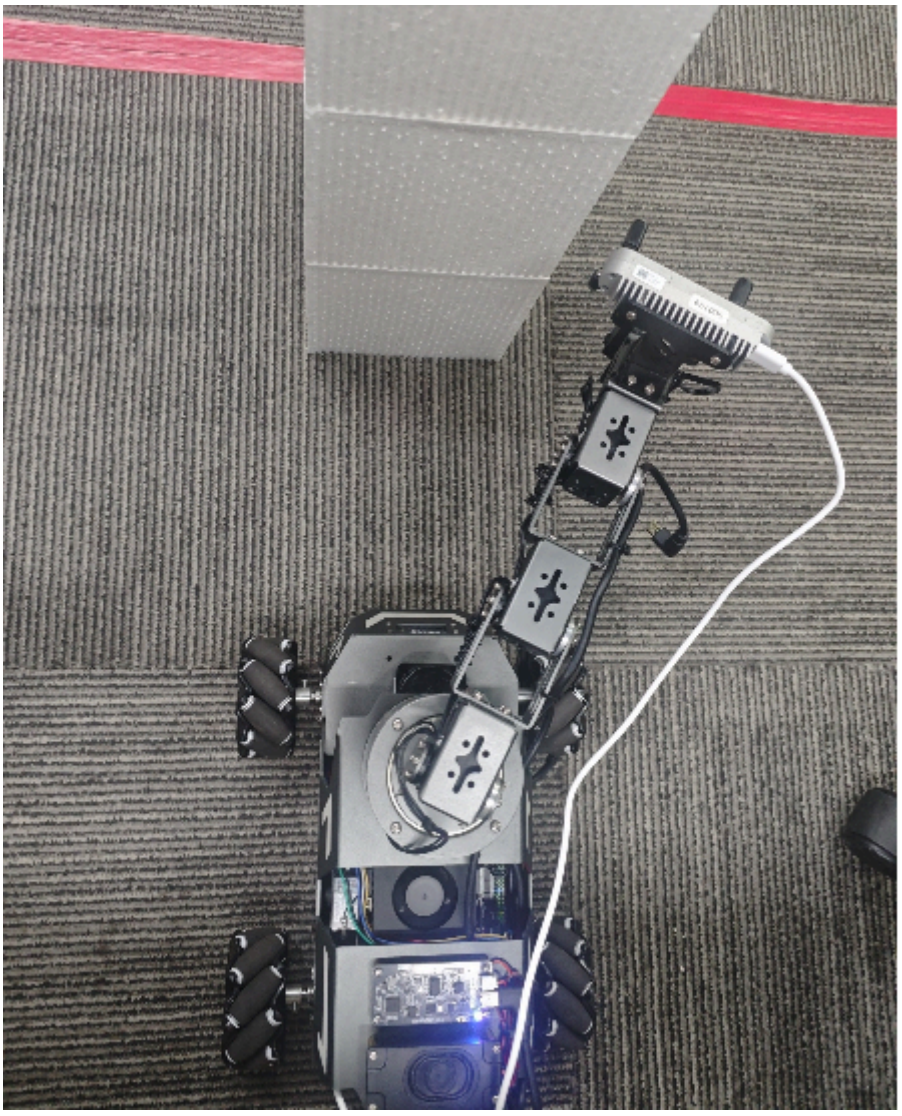
- Move to pose B

```
ros2 run moveit_demo set_target_joints --ros-args -p target_joints:="
[115.0,8.0,84.0,78.0,90.0]"
```

- The motion path planned by the robot arm will preferentially go around the obstacle.

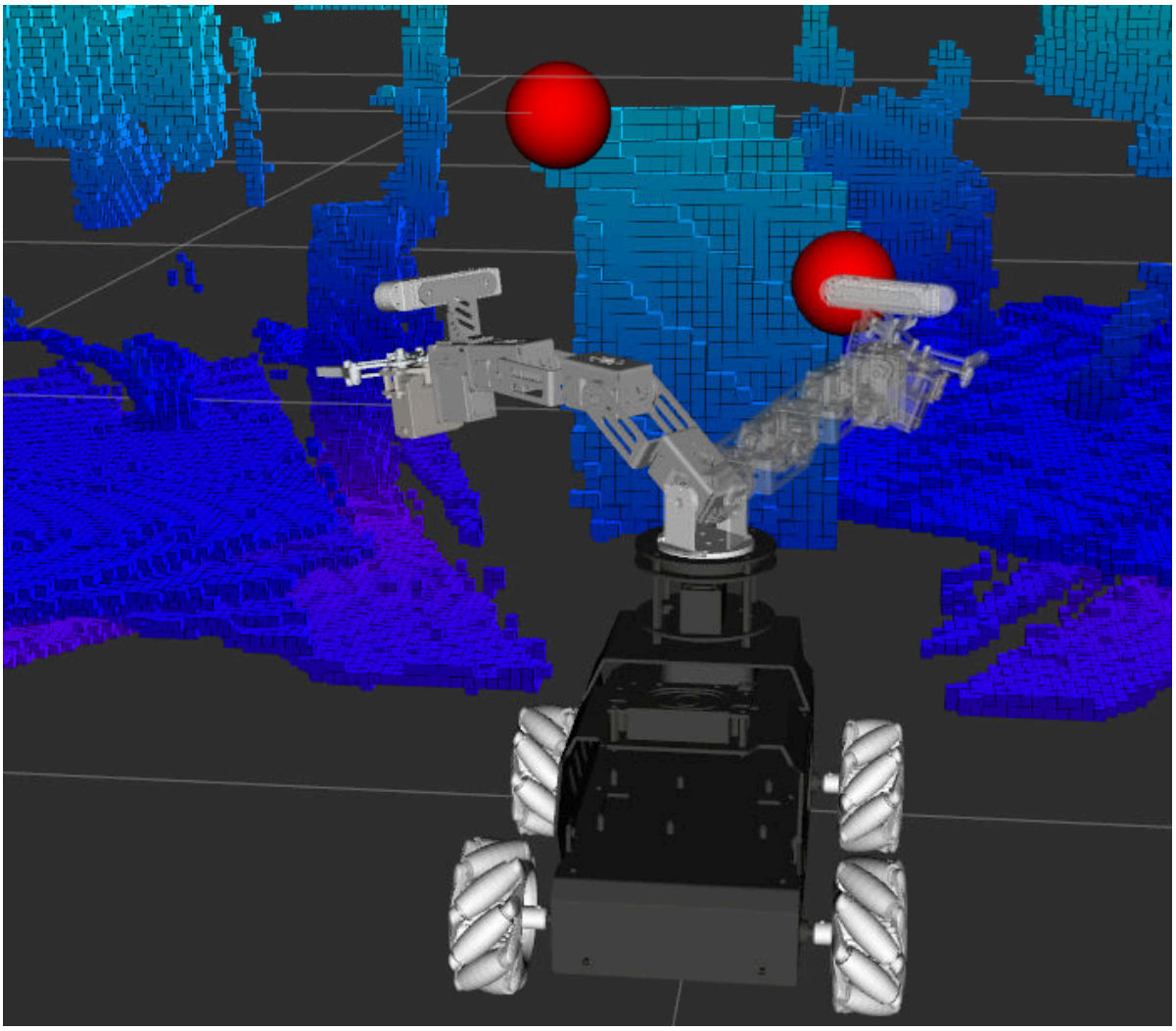






- If planning fails, it means the path planned this time will collide with the obstacle in space. RViz will show the collision point of the **predicted** trajectory as a red sphere. Just move again with `ros2 run moveit_demo set_target_joints --ros-args -p target_joints:<xxxx>` to re-plan the path.

```
[ERROR] [1789379673.275502203] [move_group_interface]: MoveGroupInterface::plan() failed or timeout reac
hed
[ERROR] [1789379673.275854992] [set_target_joints]: Planning failed!
```



4. Core Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/moveit_simulatio
n.launch.py
```

Initial pose and sensor configuration description:

- `use_3d_perception`: Checks whether the launch parameter `3d_perception` is "true", determining whether to load the 3D perception configuration.
- `init_pose_3d_deg = [90.0, 170.0, 0.0, -10.0, 90.0]`: The initial pose (degrees) in 3D perception mode; the robot arm is raised to a higher position to avoid collision with obstacles.
- In non-3D-perception mode, the default initial pose is read from the YAML configuration file.
- `deg_to_rad`: The conversion factor from degrees to radians.
- `temp_yaml_path`: Writes the initial joint angles into a temporary YAML file for MoveIt to load at startup.

MoveIt configuration building:

- `sensors_3d_real.yaml`: The path of the 3D perception sensor configuration file, containing the parameters for converting the depth camera point cloud to Octomap.

- `MoveItConfigsBuilder(...).sensors_3d(file_path=sensors_file)`: Loads the sensor configuration; it is called only in 3D perception mode, connecting the depth camera point cloud data to MoveIt's occupancy grid monitor.
- `.robot_description(mappings={"initial_positions_file": temp_yaml_path})`: Passes the initial joint angle file path into the robot description.

```
    deg_to_rad = math.pi / 180.0
    arm_names = ['arm1', 'arm2', 'arm3', 'arm4', 'arm5']

    def launch_setup(context, *args, **kwargs):
        use_3d_perception = LaunchConfiguration("3d_perception").perform(context)
== "true"

        if use_3d_perception:
            init_pose_3d_deg = [90.0, 170.0, 0.0, -10.0, 90.0]
            initial_positions = {arm_names[i]: init_pose_3d_deg[i] * deg_to_rad
for i in range(5)}
        else:
            with open(common_yaml_path, 'r') as f:
                config = yaml.safe_load(f)
            init_pose_deg = config['/**']['ros__parameters']['init_pose'][:5]
            initial_positions = {arm_names[i]: init_pose_deg[i] * deg_to_rad for
i in range(5)}

        temp_yaml_path = os.path.join(tempfile.gettempdir(),
'moveit_sim_initial_positions.yaml')
        with open(temp_yaml_path, 'w') as f:
            yaml.dump({'initial_positions': initial_positions}, f)

        if use_3d_perception:
            sensors_file =
os.path.join(get_package_share_directory("m1pro_moveit_config"), "config",
"sensors_3d_real.yaml")
            moveit_config = (
                MoveItConfigsBuilder("ROSMASTER_M1Pro",
package_name="m1pro_moveit_config")
                .sensors_3d(file_path=sensors_file)
                .robot_description(mappings={"initial_positions_file":
temp_yaml_path})
                .to_moveit_configs()
            )
        else:
            moveit_config = (
                MoveItConfigsBuilder("ROSMASTER_M1Pro",
package_name="m1pro_moveit_config")
                .robot_description(mappings={"initial_positions_file":
temp_yaml_path})
                .to_moveit_configs()
            )

        launch_package_path = get_package_share_directory("m1pro_moveit_config")
```

```
        # Robot state publisher
        rsp_node = Node(
            package="robot_state_publisher",
            executable="robot_state_publisher",
            respawn=True,
            output="screen",
            parameters=[
                moveit_config.robot_description],
        )

        # Move group
        move_group = IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(launch_package_path, "launch",
"move_group.launch.py")
            ),
            launch_arguments={
                "3d_perception": "true" if use_3d_perception else "false",
            }.items(),
        )

        ros2_control_manager = Node(
            package="controller_manager",
            executable="ros2_control_node",
            parameters=[
                os.path.join(moveit_config.package_path, "config",
"ros2_controllers.yaml"),
            ],
            remappings=[
                ("/controller_manager/robot_description", "/robot_description"),
            ],
        )

        spawn_controllers = IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(launch_package_path, "launch",
"spawn_controllers.launch.py")
            ),
        )

        static_tf = IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(get_package_share_directory("yahboomcar_bringup"),
"launch", "static_tf.launch.py")
            ),
        )
```

5. Parameter Modification

- Parameter file path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/m1pro_moveit_config/config/sensors_3d_real
.yaml
```

- The m1pro_moveit_config package uses linked compilation; saving the program after modifying the parameters will take effect.

```
octomap_resolution: 0.01
octomap_frame: measure
sensors:
  - camera_pointcloud
camera_pointcloud:
    sensor_plugin: occupancy_map_monitor/PointCloudOctomapUpdater
    point_cloud_topic: /ascamera_hp60c/camera_publisher/depth0/points
    max_range: 1.5
    point_subsample: 3
    padding_offset: 0.02
    padding_scale: 1.0
    max_update_rate: 3.0
    filtered_cloud_topic: /filtered_points
```

Parameter description:

- `octomap_resolution: 0.01`: The resolution of the Octomap voxel grid (meters); the smaller the value, the higher the obstacle detection precision, but the greater the computation.
- `octomap_frame: measure`: The reference coordinate frame of the Octomap is measure.
- `sensors`: The sensor list; here a depth camera sensor named `camera_pointcloud` is configured.
- `sensor_plugin: occupancy_map_monitor/PointCloudOctomapUpdater`: The sensor plugin type, which converts point cloud data into Octomap occupancy grid updates.
- `point_cloud_topic`: The name of the depth camera point cloud topic; subscribe to this topic to obtain the environment depth information.
- `max_range: 1.5`: The maximum effective distance of the point cloud (meters); points beyond this distance are ignored to avoid distant noise interference.
- `point_subsample: 3`: The point cloud downsampling factor; 1 out of every 3 points is taken, reducing the computation.
- `padding_offset: 0.02`: The collision object expansion offset (meters); expanding outward by 2 cm from the obstacle surface to increase the safety distance.
- `padding_scale: 1.0`: The collision object expansion scaling factor; 1.0 means no additional scaling.
- `max_update_rate: 3.0`: The maximum update frequency of the Octomap (Hz), i.e., at most 3 updates per second.
- `filtered_cloud_topic: /filtered_points`: The topic for publishing the filtered cloud, which can be used to view the adopted point cloud data in RViz.